

数据库系统期末设计

1 需求描述

本次数据库系统课程期末设计初步计划实现一个在线校园论坛系统，其主要功能包括如下几点：

- (1) 用户管理：用户注册登录，在注册时标明学生或者老师；
- (2) 板块管理：设置多个讨论板块，指定不同的主题，也可搜索指定板块下的帖子；
- (3) 帖子管理：用户可以发布、浏览帖子，进行点赞等交互，帖子支持简单的检索；
- (4) 评论管理：对帖子进行评论，支持评论回复。
- (5) 收藏管理：用户可以收藏帖子，方便在自己的收藏夹里快速找到。

整体而言，基础功能类似于现在的《校园集市》、《小红书》、《百度贴吧》等典型的论坛平台，后续可能在此基础上进行其余功能的扩充与完善。

2 ER 模型设计

2.1 实体及属性

本设计包含以下实体：

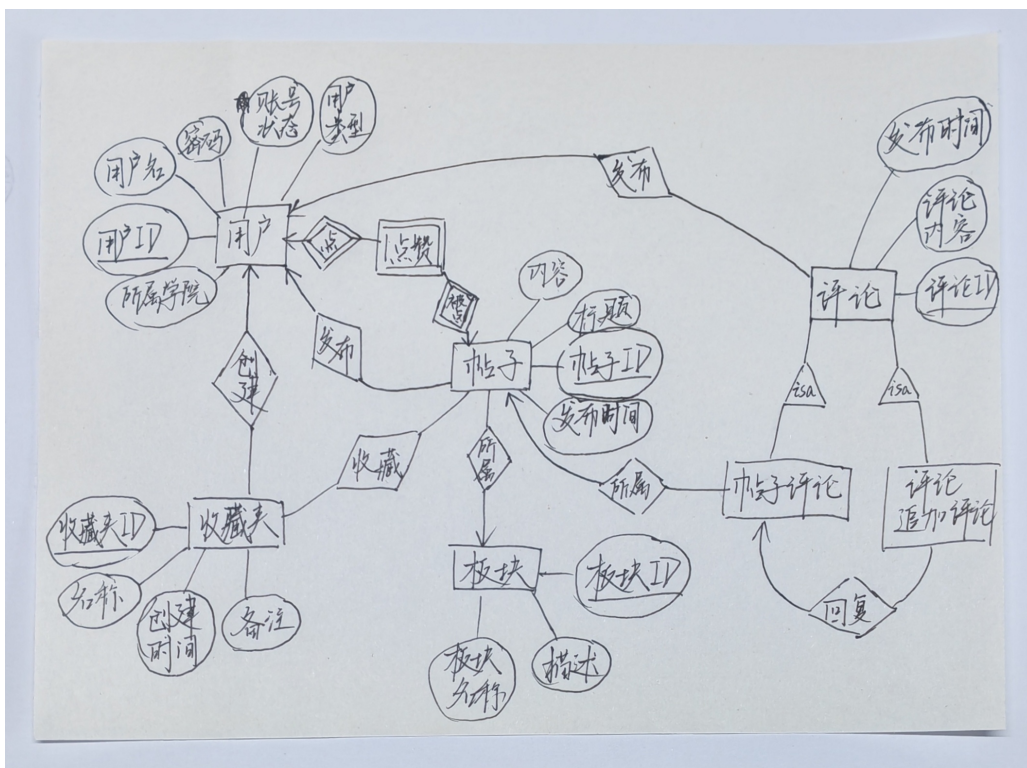
1. **用户 (User)** 属性：用户 ID (主键)、用户名、密码、账号状态、用户类型 (区分学生/老师)，所属学院 (可选)。
2. **板块 (Section)** 属性：板块 ID (主键)、板块名称、板块描述。
3. **帖子 (Post)** 属性：帖子 ID (主键)、标题、内容、发布时间、发布人 ID (外键)、所属板块 ID (外键)。
4. **评论 (Comment, 超类)** 属性：评论 ID (主键)、评论内容、发布时间、发布人 ID (外键)。

子类：

- 帖子评论 (CommentOfPost): 额外属性: 所属帖子 ID (外键);
 - 评论追加评论 (CommentOfComment): 额外属性: 父评论 ID (外键)。
5. **点赞 (PostLike)** 属性: 用户 ID (外键)、帖子 ID (外键)、点赞时间, 联合主键为用户 ID+ 帖子 ID (每个人每个帖子只能点赞一次)。
 6. **收藏夹 (FavoriteFolder)** 属性: 收藏夹 ID (主键)、用户 ID (外键)、收藏夹名称、创建时间、备注。

2.2 ER 图与关系模式

ER 图:



关系模式: (主键用下划线标注, 外键用 $\triangle$ 标注)

1. **User**(user_id, username, password, status, user_type, college)
2. **Section**(section_id, section_name, description)
3. **Post**(post_id, title, content, publish_time, $\triangle$ user_id, $\triangle$ section_id)
4. **Comment**(comment_id, content, publish_time, $\triangle$ user_id)
5. **CommentOfPost**($\triangle$ comment_id, $\triangle$ post_id)
6. **CommentOfComment**($\triangle$ comment_id, $\triangle$ parent_comment_id)

7. **PostLike**(Δ user_id, Δ post_id, like_time)

8. **FavoriteFolder**(favorite_id, Δ user_id, folder_name, create_time, remark)

3 SQL 建表语句

```
1  -- 1. 用户表
2  CREATE TABLE User (
3      user_id INT PRIMARY KEY AUTO_INCREMENT,
4      username VARCHAR(50) NOT NULL UNIQUE,
5      password VARCHAR(100) NOT NULL,
6      status TINYINT DEFAULT 1, -- 1正常 0禁用
7      user_type TINYINT NOT NULL, -- 1学生 2老师
8      college VARCHAR(50)
9  );
10
11 -- 2. 板块表
12 CREATE TABLE Section (
13     section_id INT PRIMARY KEY AUTO_INCREMENT,
14     section_name VARCHAR(50) NOT NULL UNIQUE,
15     description TEXT
16 );
17
18 -- 3. 帖子表
19 CREATE TABLE Post (
20     post_id INT PRIMARY KEY AUTO_INCREMENT,
21     title VARCHAR(100) NOT NULL,
22     content TEXT NOT NULL,
23     publish_time DATETIME DEFAULT CURRENT_TIMESTAMP,
24     user_id INT NOT NULL,
25     section_id INT NOT NULL,
26     FOREIGN KEY (user_id) REFERENCES User(user_id),
27     FOREIGN KEY (section_id) REFERENCES Section(section_id)
28 );
29
30 -- 4. 评论超类表
31 CREATE TABLE Comment (
32     comment_id INT PRIMARY KEY AUTO_INCREMENT,
33     content TEXT NOT NULL,
34     publish_time DATETIME DEFAULT CURRENT_TIMESTAMP,
```

```
35     user_id INT NOT NULL,
36     FOREIGN KEY (user_id) REFERENCES User(user_id)
37 );
38
39 -- 5. 帖子评论 (评论子类)
40 CREATE TABLE CommentOfPost (
41     comment_id INT PRIMARY KEY,
42     post_id INT NOT NULL,
43     FOREIGN KEY (comment_id) REFERENCES Comment(comment_id),
44     FOREIGN KEY (post_id) REFERENCES Post(post_id)
45 );
46
47 -- 6. 评论追加评论 (评论子类)
48 CREATE TABLE CommentOfComment (
49     comment_id INT PRIMARY KEY,
50     parent_comment_id INT NOT NULL,
51     FOREIGN KEY (comment_id) REFERENCES Comment(comment_id),
52     FOREIGN KEY (parent_comment_id) REFERENCES Comment(comment_id)
53 );
54
55 -- 7. 点赞表 (联合主键, 唯一约束)
56 CREATE TABLE PostLike (
57     user_id INT NOT NULL,
58     post_id INT NOT NULL,
59     like_time DATETIME DEFAULT CURRENT_TIMESTAMP,
60     PRIMARY KEY (user_id, post_id),
61     FOREIGN KEY (user_id) REFERENCES User(user_id),
62     FOREIGN KEY (post_id) REFERENCES Post(post_id)
63 );
64
65 -- 8. 收藏夹表
66 CREATE TABLE FavoriteFolder (
67     favorite_id INT PRIMARY KEY AUTO_INCREMENT,
68     user_id INT NOT NULL,
69     folder_name VARCHAR(50) NOT NULL,
70     create_time DATETIME DEFAULT CURRENT_TIMESTAMP,
71     remark VARCHAR(100),
72     FOREIGN KEY (user_id) REFERENCES User(user_id)
73 );
```

4 查询语句样例

4.1 单表查询

查询所有正常状态的学生用户

```
1 SELECT user_id, username, college FROM User WHERE user_type=1 AND  
   status=1;
```

4.2 多表连接查询

查询帖子标题、作者、所属板块及点赞数

```
1 SELECT p.title, u.username, s.section_name, COUNT(l.user_id) AS  
   like_count  
2 FROM Post p  
3 JOIN User u ON p.user_id = u.user_id  
4 JOIN Section s ON p.section_id = s.section_id  
5 LEFT JOIN PostLike l ON p.post_id = l.post_id  
6 GROUP BY p.post_id;
```

4.3 嵌套子查询

查询创建过收藏夹的用户信息

```
1 SELECT user_id, username, user_type FROM User  
2 WHERE user_id IN (SELECT DISTINCT user_id FROM FavoriteFolder);
```

4.4 EXISTS 查询

查询被点赞过的帖子

```
1 SELECT post_id, title, publish_time FROM Post p  
2 WHERE EXISTS (SELECT 1 FROM PostLike l WHERE l.post_id = p.post_id);
```

4.5 聚合分组查询

统计每个板块的帖子总数

```
1 SELECT s.section_name, COUNT(p.post_id) AS post_count  
2 FROM Section s  
3 LEFT JOIN Post p ON s.section_id = p.section_id
```

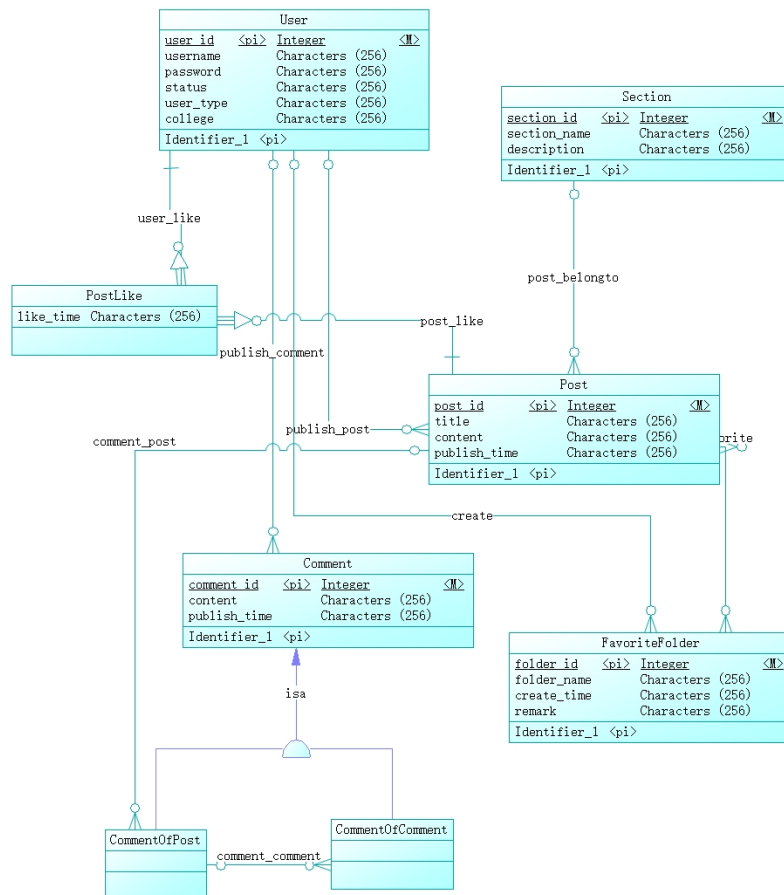
```

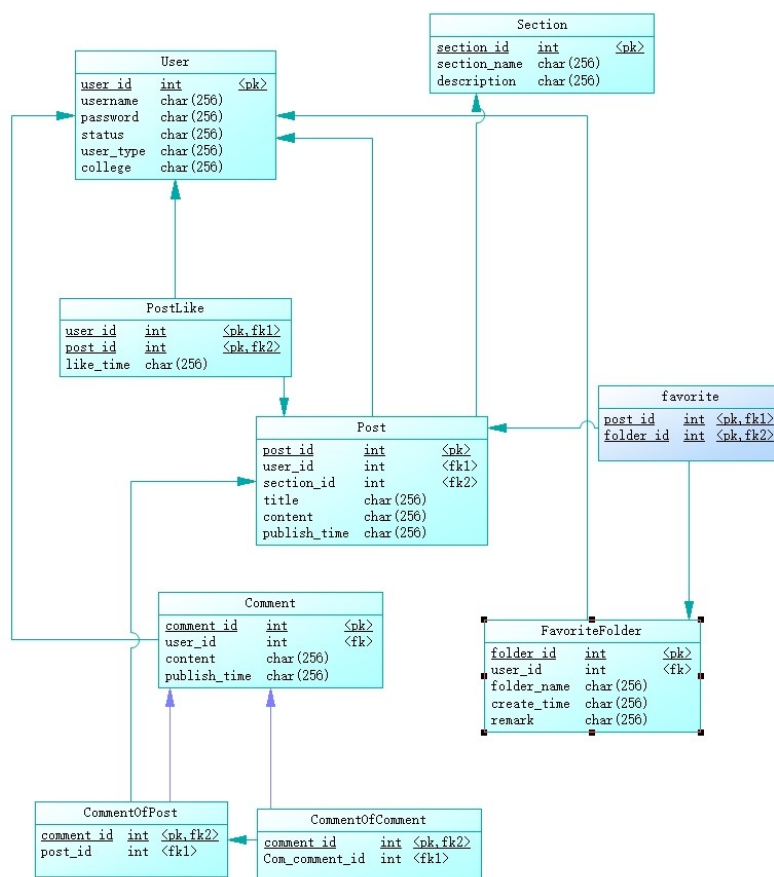
4 GROUP BY s.section_id
5 ORDER BY post_count DESC;

```

4.6 PowerDesigner 设计 ER 图

使用 PowerDesigner 实现上述实体关系得到以下模型：





4.7 PowerDesigner 生成 SQL 语句

使用 PowerDesigner 建立数据库的功能，自动生成得到如下语句：

```

1  /*=====*/
2  /* DBMS name:      MySQL 5.0 */
3  /* Created on:     2026/4/12 14:43:02 */
4  /*=====*/
5
6
7  drop table if exists Comment;
8
9  drop table if exists CommentOfComment;
10
11 drop table if exists CommentOfPost;
12
13 drop table if exists FavoriteFolder;
14
15 drop table if exists Post;
16
17 drop table if exists PostLike;
  
```

```
18
19 drop table if exists Section;
20
21 drop table if exists User;
22
23 drop table if exists favorite;
24
25 /*=====*/
26 /* Table: Comment */
27 /*=====*/
28 create table Comment
29 (
30     comment_id          int not null,
31     user_id             int,
32     content             char(256),
33     publish_time        char(256),
34     primary key (comment_id)
35 );
36
37 /*=====*/
38 /* Table: CommentOfComment */
39 /*=====*/
40 create table CommentOfComment
41 (
42     comment_id          int not null,
43     Com_comment_id      int,
44     primary key (comment_id)
45 );
46
47 /*=====*/
48 /* Table: CommentOfPost */
49 /*=====*/
50 create table CommentOfPost
51 (
52     comment_id          int not null,
53     post_id             int,
54     primary key (comment_id)
55 );
56
57 /*=====*/
```

```

58  /* Table: FavoriteFolder */
59  /*=====*/
60  create table FavoriteFolder
61  (
62      folder_id          int not null,
63      user_id            int,
64      folder_name        char(256),
65      create_time        char(256),
66      remark             char(256),
67      primary key (folder_id)
68  );
69
70  /*=====*/
71  /* Table: Post */
72  /*=====*/
73  create table Post
74  (
75      post_id            int not null,
76      user_id            int,
77      section_id         int,
78      title              char(256),
79      content            char(256),
80      publish_time       char(256),
81      primary key (post_id)
82  );
83
84  /*=====*/
85  /* Table: PostLike */
86  /*=====*/
87  create table PostLike
88  (
89      user_id            int not null,
90      post_id            int not null,
91      like_time          char(256),
92      primary key (user_id, post_id)
93  );
94
95  /*=====*/
96  /* Table: Section */
97  /*=====*/

```

```
98 create table Section
99 (
100     section_id          int not null,
101     section_name        char(256),
102     description         char(256),
103     primary key (section_id)
104 );
105
106 /*=====*/
107 /* Table: User */
108 /*=====*/
109 create table User
110 (
111     user_id             int not null,
112     username            char(256),
113     password            char(256),
114     status              char(256),
115     user_type           char(256),
116     college             char(256),
117     primary key (user_id)
118 );
119
120 /*=====*/
121 /* Table: favorite */
122 /*=====*/
123 create table favorite
124 (
125     post_id             int not null,
126     folder_id           int not null,
127     primary key (post_id, folder_id)
128 );
129
130 alter table Comment add constraint FK_publish_comment foreign key (
131     user_id)
132     references User (user_id) on update restrict;
133
134 alter table CommentOfComment add constraint FK_comment_comment
135     foreign key (Com_comment_id)
136     references CommentOfPost (comment_id) on update restrict;
```

```
136 alter table CommentOfComment add constraint FK_isa2 foreign key (
      comment_id)
137       references Comment (comment_id) on update restrict;
138
139 alter table CommentOfPost add constraint FK_comment_post foreign key
      (post_id)
140       references Post (post_id) on update restrict;
141
142 alter table CommentOfPost add constraint FK_isa foreign key (
      comment_id)
143       references Comment (comment_id) on update restrict;
144
145 alter table FavoriteFolder add constraint FK_create foreign key (
      user_id)
146       references User (user_id) on update restrict;
147
148 alter table Post add constraint FK_post_belongto foreign key (
      section_id)
149       references Section (section_id) on update restrict;
150
151 alter table Post add constraint FK_publish_post foreign key (user_id)
152       references User (user_id) on update restrict;
153
154 alter table PostLike add constraint FK_post_like foreign key (post_id
      )
155       references Post (post_id) on update restrict;
156
157 alter table PostLike add constraint FK_user_like foreign key (user_id
      )
158       references User (user_id) on update restrict;
159
160 alter table favorite add constraint FK_favorite foreign key (post_id)
161       references Post (post_id) on update restrict;
162
163 alter table favorite add constraint FK_favorite2 foreign key (
      folder_id)
164       references FavoriteFolder (folder_id) on update restrict;
```

5 设计分析与比较

5.1 两种关系模式的设计差异及影响

从代码上来看，PowerDesigner 生成的语句在主键与外键描述上较为零散，代码可读性较差；在子类继承方面，生成语句时自动选择的是空值继承，需要手动修改生成选项，才能变为 ER 类型的类表继承；两者实现虽然一样，但是在语言风格上差异较大：PowerDesigner 生成的语句仅实现最基础的功能，可以作为项目测试时快速实现逻辑，但在后期工程上，还需在此基础上进行大量的改进调整与优化，才能使其更加贴合实际业务。

除此之外，PowerDesigner 上的数据类型也较为有限，在某些特定属性上难以精准描述。虽然可以用其他等效的类型替代，但其仍为项目构建带来了部分底层上的影响，一定程度上增添了代码调试的工程量。

5.2 PowerDesigner 生成 SQL 语句的特点

在语句特点上，PowerDesigner 生成的 SQL 语句具有标准化模板化的特征，固定包含标注 DBMS 类型、生成时间的文件头注释，格式统一且符合工程化规范；同时，输出流程固定，均为先通过 `drop table if exists` 删除旧表避免冲突，再执行 `create table` 建表，最后用 `alter table` 添加外键约束。这样的标准化语句建表虽然规范了代码结构，但是可读性相对较差，且容易与实际业务逻辑脱节。

生成的附加语句主要包括 `drop table` 删除语句、`alter table` 外键约束语句及冗余表和字段，其成因各不相同：`drop table` 语句是为保障建表幂等性，避免重复建表报错；`alter table` 约束语句源于工具分层构建物理模型的机制，先建基础表结构，再单独添加约束；冗余表和字段则是工具按 ER 图默认规则自动生成，与在画板上设计出的 ER 图具有强制的关联。在作用上，`drop table` 语句解决表的重复创建问题，`alter table` 约束语句实现表结构与约束分离，保障数据库的参照完整性；冗余中间表和字段虽与业务逻辑不符，但可作为后期工程优化与功能添加的基础，如果确定确实没有必要，也可以通过相关配置关闭冗余以规范代码。